VU2CPL Shack Changelog

Fixes and changes to non-DXCC tabs of the Node-RED shack automation (SPE amplifier, Power Control, Solar Conditions, Lightning, Rotator, etc.).

The DXCC Tracker has its own doc: see `README.md` / `DXCC_Tracker_README.pdf`.

2026-04-22

SPE Amplifier — Power meter scales with selected power level

Tab: SPE (`648eb83c2566c7b6`) **Node:** SPE → Dashboard Formatter (`8f7d96af1a4c24a9`)

The output-power bar was always scaled to a 1500 W full-scale regardless of the amp's selected power level. It now auto-scales based on `col10` (L / M / H):

Level	Full scale
L (Low)	500 W
M (Middle)	1000 W
H (Maximum)	1500 W

The bar text, percentage width, and amber/red thresholds (70 % / 90 %) all follow the new max automatically. Dashboard template (`e5b15d8e700bf002`) was already reading `d.pwrMax` — no UI change needed.

Power Control Panel — Fix stale "ON" defaults + reliable state refresh

Tab: All Power Strips (`b76a5310767803b4`)

Three related bugs resolved:

1. **HTML defaults lied.** Rotator, USB2, and USB3 plug tiles had `class="pw-plug on"` hardcoded in the template (`780b75182df31634`) — they showed ON on every page load until an MQTT `stat/.../POWER<n>` arrived. Since Tasmota only publishes `stat/.../POWER<n>` on a *state change*, the stale classes could persist indefinitely. All three now default to `off`.

2. **Dashboard ignored Tasmota's query responses.** `Power State → Dashboard` (`f8c3c072b381bd1c`) was filtering out `/RESULT` topics entirely, so responses to state queries (which Tasmota delivers on `stat/<device>/RESULT` as `{"POWER<n>":"..."}`) never reached the panel. The function now parses the RESULT JSON and extracts every `POWER<n>` key into the shared `power_states` object.

3. **Startup poll only queried POWER1.** The four `Poll <device>` function nodes (powerstrip1, powerstrip2, powerstrip3, 4relayboard) emitted a single empty-payload query to `cmnd/<device>/Power`, which Tasmota interprets as a POWER1 query only. They now loop over every outlet on the device, emitting one query per POWER:

```
for (var i = 1; i <= 5; i++) {
    node.send({ topic: 'cmnd/<device>/POWER' + i, payload: '' });
```

```
}
```

Combined with fix #2, this means every 30 s (and on deploy) the dashboard receives a fresh state for every outlet — the "always on" bug cannot persist more than ~30 s.

Solar Conditions — Geomagnetic K/A column indices

Tab: Solar (`590e889d44815afb`) **Node:** Parse K and A (`6e22622d54f653b1`)

The daily geomagnetic indices text file from NOAA (`services.swpc.noaa.gov/text/daily-geomagnetic-indices.txt`) has three index groups per data row:

Columns	Group
3	Middle-Latitude A
4–11	Middle-Latitude 8 × K
12	High-Latitude A
13–20	High-Latitude 8 × K
21	**Planetary A (Ap)**
22–29	**Planetary 8 × Kp (fractional)**

The parser was reading `cols[19]` for Ap and scanning `cols[20]..cols[27]` for Kp — that's the high-latitude K range plus the first six planetary Kp values, off by two columns in every case. Corrected to `cols[21]` for Ap and `cols[22]..cols[29]` for Kp. The length guard was also bumped from `< 28` to `< 30` .

Observed effect: yesterday's real values (2026-04-21) are A = 19, K ≈ 2.00. Before the fix the dashboard was showing A ≈ 3, K ≈ 4.00.

Solar Conditions — Aggregator branch discriminators

Tab: Solar (`590e889d44815afb`) **Node:** Solar State Aggregator (`d8ed9693432963df`)

The aggregator's SFI, K-Index, and A-Index branches all accepted any numeric `msg.payload` with a `msg.label` . In practice:

- The **SFI branch** matched every K / A message and happened to survive only because the SFI HTTP response arrived *last* in each poll cycle.
- The **K branch** had no discriminator against A, so the A-Index message (second output of Parse K and A) would overwrite `st.k` with the A value.

This was invisible while the parser was producing bogus near-identical K and A values (pre-column-fix). Once the column-index fix started producing a correct A = 19, the K gauge began showing 19.0 with the A's "Unsettled" label.

The three branches now discriminate by their gauge's `control.sectors` shape:

Metric	Discriminator

SFI	`sectors[0].val === 50` (gauge min is 50, not 0)
K	`sectors[0].val === 0.0 && sectors[1].val < 15` (K's 4.5)
A	`sectors[0].val === 0.0 && sectors[1].val > 10` (A's ≥ 35.5)

Rotator Auto-Off Timer — 60 s countdown reset loop

Tab: All Power Strips (`b76a5310767803b4`) **Node:** Rotator Auto-Off Timer (`05f0ddeb566a90fc`)

Symptom: rotator turned on, 60 s timer counted down to 0, then reset itself back to 60 s; physical switch stayed ON indefinitely. Cause: the timer unconditionally restarts whenever a `stat/powerstrip1/POWER2` = `"ON"` message arrives, and something (Tasmota echo, state-poll response, possibly a SetOption-related republish) keeps sending `"ON"` for an already-on outlet.

Fix: make the timer **idempotent** and add a post-OFF cooldown:

- If a timer is already running, additional `"ON"` messages are ignored (they do not reset the countdown).
- After the timer fires OFF (or an OFF is received), a 10 s cooldown begins. Any `"ON"` during that window is ignored — this breaks the reset loop regardless of its source.

Also fixed the status string from "Timer running — 5 min" to "Timer running — 60 s" (it has always been 60 s in code; the label was stale from an earlier intended value).

2026-05-01

AS3935 Lightning Sensor — Service revived after 10-day silent outage

Service: `as3935.service` (systemd) → `/home/vu2cpl/as3935_mqtt.py`

The lightning sensor daemon had been **dead since 2026-04-21 09:14 IST** and went unnoticed for 10 days because the dashboard had no liveness indicator. Discovered when a real storm fired no MQTT events.

Root cause of the original crash: at boot, `paho-mqtt`'s `client.connect()` called before the network was ready and threw `OSError: [Errno 101] Network is unreachable`. systemd hit `StartLimitBurst` and gave up. The script had no retry loop and no liveness signal, so silence looked identical to "no nearby strikes."

Hardening applied to `as3935_mqtt.py`

Change	Why
`mqtt_connect_with_retry()` (2 → 60 s exponential backoff)	Survives boot-time network race
Last Will & Testament on `lightning/as3935/status`	Broker auto-publishes `offline` if script dies
Startup retained publish to `lightning/as3935/status` (`event:"ready"`)	Visible in MQTT Explorer immediately
30-second heartbeat to `lightning/as3935/hb` (retained, includes `uptime_s` + per-event counters)	Silence > 60 s now unambiguously means dead

`on_connect` / `on_disconnect` callbacks log broker state	Easier diagnosis
I²C self-test on startup — reads `REG_CFG0` and logs the value	Catches dead-bus cases before the IRQ loop
Clean SIGTERM handler — publishes `offline` + GPIO cleanup	systemd `stop` no longer leaves a stuck GPIO claim
`try/except` around IRQ handler body	One bad I²C read can't crash the daemon
Event counters (lightning / disturber / noise / irq) in heartbeat	Observable from MQTT Explorer without reading logs

Antenna mode parametrization

The original script's `set_outdoor_mode()` wrote `0x0C` to `REG_CFG0`, which actually maps to `AFE_GB = 0x06` — *less* sensitive than the datasheet's "outdoor" value (`AFE_GB = 0x0E` → write `0x1C`). For the current indoor antenna placement, gain should be higher still: `AFE_GB = 0x12` → write `0x24`.

Replaced with `set_antenna_mode("indoor" | "outdoor")` controlled by a top-of-file `ANTENNA_LOCATION` constant. Currently `"indoor"` since the ferrite antenna is indoors. The mode is also published in the retained status payload so the dashboard can surface it.

Note: an indoor AS3935 has effective range of a few km at most (walls block the H-field) and is prone to false disturbers from local RF. Open-Meteo CAPE is the primary detection layer until the antenna is moved outdoors.

Required systemd unit override

`Environment=PYTHONUNBUFFERED=1` must be in `/etc/systemd/system/as3935.service.d/override.conf` — without it, `print()` output is block-buffered and never reaches `journalctl`. This is what made the running-but-silent state indistinguishable from "crashed."

Lightning Antenna Protector — Open-Meteo: `lightning_potential` is null in IN

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Build Open-Meteo URL (`3d61a4f561d40c70`), Parse Open-Meteo → Strike (`593f22a507b46335`)

Symptom: dashboard's OPEN-METEO MONITOR badge stuck at "Waiting for data…" indefinitely; auto-disconnect never fired even during real storms.

Root cause: the flow polled the `lightning_potential` parameter, which Open-Meteo's ICON-Global model returns as **all- `null` for India** (verified: 0/24 non-null entries for 13.065°N 77.806°E). Every poll hit `index === null`, the parser silently returned `null`, and nothing reached the dashboard or strike chain.

A second issue compounded it: the parser's mapping `km = (1 - index/100) × 200` assumed `lightning_potential` was a 0–100 percentage. Open-Meteo actually returns it as J/kg (LPI, an ICON-model quantity that typically ranges 0–25 even for severe storms). Even if the data had been present, the formula would have mapped a 25 J/kg severe storm to 150 km — well above any practical threshold — so disconnection would still never trigger.

Fix

Build Open-Meteo URL now requests `cape`, `weather_code`, and `precipitation_probability` (hourly + current weather_code). All three have continuous coverage globally.

Parse Open-Meteo → Strike rewritten with two outputs:

Decision	Synthetic km
`current.weather_code ∈ {95, 96, 99}` (TS now)	0 (overhead)
Hourly `weather_code ∈ {95, 96, 99}`	0
Showers (80–82) + CAPE ≥ 1000	20
CAPE ≥ 2500	10 (severe)
CAPE ≥ 1500	40 (strong)
CAPE ≥ 800	100 (moderate)
else	calm — no strike emitted

Output 1 → existing Parse Strike chain (only when storm-relevant). Output 2 → reserved for Master Dashboard live status (wire pending).

CAPE is *predictive* — it catches the convective setup hours before the storm peaks, which is what we want for a pre-emptive antenna disconnect. weather_code 95/96/99 is *deterministic* — when the model reports actual thunderstorm at the grid cell, force overhead.

Lightning Antenna Protector — Strike source label propagation

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Parse Strike (`26ddff0cbbfe5fc1`), Trigger Disconnect (`d62fb0c3c40f03b7`)

Event log lines for Open-Meteo strikes were appearing as `DISCONNECT: Blitzortung strike N km` because:

1. **Parse Strike Case 1** (object payloads from Parse Open-Meteo and test injects) constructed `msg.strike` without copying the upstream `payload.source` field. Source was lost on every object-shaped strike.
2. **Trigger Disconnect** had a fallback default of `'Blitzortung'` when `msg.strike.source` was missing — a leftover from when Blitzortung TCP was the original real-time source (Cases 2 and 3 in Parse Strike, which parse a binary/string TCP feed and are currently dead code — no upstream node feeds them).

Fix

```
// Parse Strike Case 1
msg.strike = { lat, lon, time, pol, region, source: d.source };  // added source

// Trigger Disconnect
const src = msg.strike && msg.strike.source ? msg.strike.source : 'unknown';
```

Cases 2/3 left intact — if Blitzortung TCP is wired in later, set `source: 'Blitzortung'` there at the same time.

Verified: log now reads `DISCONNECT: Open-Meteo strike 0 km`.

Lightning Antenna Protector — Dashboard refresh persistence

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes:** Strike → Dashboard (`51367d456e71fb3e`), Master Dashboard (`557083037f168b22`), Refresh Stats (`61dca3d98a0e4c28`), Clear map every 30 min (`55f94d9dde0dc893`), new node Strikes Replay, new node Clear All

Symptom: refreshing the dashboard browser tab wiped the **event log** and **strike map markers** until the next event arrived. Both data sets were held only in dashboard-local JS variables (`var strikes=[]` and the `logLines` innerHTML), with no server-side persistence or replay path.

Fix — persist on the flow side, replay on the existing 30 s tick

1. **Strike → Dashboard** now appends each emitted strike to a flow-context array (`flow.strikes`), capped at 800 entries (FIFO). The `Format Log` node was already persisting log lines to `flow.event_log`.
2. **New `Strikes Replay` function node** — reads `flow.strikes` and emits `{type:'strikes_replay', list:[...]}` to Master Dashboard.
3. **`Refresh Stats`** (the existing 30 s ticker) now also fans out to:
   - `Log → Dashboard` (re-emits cached event log)
   - `Strikes Replay` (re-emits cached map markers)
4. **Master Dashboard** picks up a new message type:

```
if (d.type === 'strikes_replay') {
    strikeLayer.clearLayers();
    strikes = [];
    d.list.forEach(s => {
        strikes.push(s);
        L.circleMarker([s.lat, s.lon], {
            radius: 5, color: s.color, fillColor: s.color,
            fillOpacity: 0.9, weight: 1
        }).addTo(strikeLayer);
    });
    document.getElementById('lzCnt').textContent = 'Strikes: ' +
strikes.length;
    return;
}
```

5. **New `Clear All` function** sits between the 30-min clear-map inject and Master Dashboard:

```
flow.set('strikes', []);
flow.set('event_log', []);
return { payload: { type: 'clear' } };
```

Clears both the server cache and the dashboard simultaneously, so the next replay tick doesn't re-paint old strikes onto a freshly cleared map.

Behavior after fix

Action	Result
Page refresh	Log + strike markers re-appear within 30 s
New strike	Immediate update; also persisted to `flow.strikes`
30-min auto-clear	Map cleared on dashboard *and* server cache
Node-RED restart	Flow context lost (default). Add `flow.context.persistent=true` for cross-restart durability — separate change.

2026-05-06

AS3935 — IRQ pin was GPIO17 in software, GPIO4 in hardware

Script: `as3935_mqtt.py` **Helper:** `as3935_tune.py` (new)

Symptom after the May 1 revival: heartbeat counters stayed at `{lightning:0, disturber:0, noise:0, irq:0}` indefinitely. MQTT, I²C, and `status: "ready"` all worked, but no IRQ events ever fired.

Root cause

The original script declared `IRQ_PIN = 17`, but the AS3935 board's INT line was physically wired to **GPIO4** (Pi physical pin 7). Since the chip's I²C path was healthy, every diagnostic short of an end-to-end IRQ test passed. The mismatch had been there since first deploy; it was hidden because the chip-self-test only exercised I²C.

How it was found

A two-step diagnostic:

1. **Pull-test on GPIO17** — with the as3935 service stopped, sampled GPIO17 with `PUD_DOWN` / `PUD_OFF` / `PUD_UP`. Pin tracked the pull setting → genuine floating input → not connected to anything.
2. **SRCO scan** — set `DISP_SRCO=1` in chip register `0x08[6]` to route the chip's internal RC oscillator onto whatever pin its INT line is bonded to. Polled every accessible Pi GPIO (4, 5, 6, ..., 27. for transitions over 200 ms. **GPIO4 showed 12,162 transitions; every other pin showed 0.** Conclusive.

`IRQ_PIN = 4` change applied. Heartbeat counters started incrementing on the next NF=0 test (see below).

AS3935 — LC tank tuning to 500 kHz

Helper: `as3935_tune.py` (new)

The AS3935 has 16 internal tuning capacitor settings (0–15, ~8 pF per step) at register `0x08[3:0]`. The script wasn't using any of them — it relied on whatever stray capacitance the antenna+wiring happened to give. For accurate sferic detection, the tank should resonate at 500 kHz ± 3.5 %.

`as3935_tune.py`

New helper that:

1. Configures `LCO_FDIV = 3` (÷128 divider — slow enough for Python event-detection to count edges reliably; ÷16 at ~31 kHz overruns the user-space callback)
2. Sweeps `TUN_CAP` 0..15, samples 2 s of edges per setting, calculates frequency = `edges × 128 / 2 s`
3. Reports the table + recommends the cap value closest to 500 kHz
4. Flags out-of-spec results (>±3.5 %) with diagnostic hints

Must be run with `as3935.service` stopped (otherwise the daemon holds GPIO4) and the antenna in its **final mounting position** — moving the antenna afterwards changes stray capacitance and invalidates the tuning.

Result for current installation

TUN_CAP	pF	Hz	err %
0	0	516,480	+3.30
9	72	501,504	+0.30
10	**80**	**499,904**	**−0.02**
11	88	498,560	−0.29
15	120	493,056	−1.39

All 16 cap settings happened to be within spec (typical of a clean antenna circuit); `TUN_CAP=10` lands at −0.02 % err. Baked into the script as `TUN_CAP = 10`.

AS3935 — Add `CALIB_RCO`, INT flush, retained-status enrichment

Script: `as3935_mqtt.py`

Three additions to the chip-init sequence in the daemon, all chasing the same goal of making "is the chip happy?" answerable from MQTT without journal grep:

1. **`CALIB_RCO` at every startup** — the AS3935 has internal TRCO and SRCO oscillators that benefit from a one-time post-power-on calibration. The original script never called it; some chips work without, but timing-sensitive code paths can misbehave. Added: write `0x96` to register `0x3D`, wait 5 ms, verify `CALIB_DONE` (bit 7) set and `CALIB_NOK` (bit 6) clear in both `0x3A` (TRCO) and `0x3B` (SRCO). Result logged + published.
2. **Pending-INT flush after configuration writes** — register writes during init can transiently spike the IRQ line. Reading `0x03` (INT) clears it. Without this flush, the first real event sometimes arrived as a no-op rising edge that the `add_event_detect` callback then ignored because INT bits read 0.
3. **Retained `lightning/as3935/status` payload now includes** `tun_cap`, `irq_pin`, `calib_trco`, `calib_srco`. Status is re-published *after* calibration completes so the retained message reflects the actual result (the `on_connect` callback fires before init finishes).

Final init log

```
[init] AS3935 CFG0=0x24 (i2c bus 1 addr 0x03)
[mqtt] connected to 192.168.1.169:1883
```

```
[init] AFE_GB set for indoor antenna (CFG0=0x24)
[init] TUN_CAP set to 10 (80 pF)
[init] CALIB_RCO  TRCO=OK (0xA3)  SRCO=OK (0xA5)
[init] Cleared pending INT: 0x0
AS3935 ready — interrupt mode on GPIO4, noise_floor=4, antenna=indoor, tun_cap=10
```

Field verification

During the May 1 storm window (NF=0 momentarily), the chip caught **209 noise events** in roughly 30 s, then went silent as the storm moved off — the expected behaviour for an indoor narrow-band 500 kHz antenna. Subsequent tests with NF=2/NF=4 in clear conditions remained at 0 events: not a bug, just no nearby sferics for the selectively-tuned tank to resonate with. Until the antenna is moved outdoors, AS3935 is a confirmation-only secondary; Open-Meteo CAPE is the primary detection layer.

Lightning Antenna Protector — AS3935 dashboard liveness panel

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Nodes added:** AS3935 Status (mqtt in), AS3935 Heartbeat (mqtt in), Format AS3935 State (function), Replay AS3935 State (function) **Node modified:** Master Dashboard (`557083037f168b22`) — three handler inserts inside `scope.$watch('msg', …)`

Symptom: even with the AS3935 daemon healthy and publishing `lightning/as3935/status` (retained, on connect) and `lightning/as3935/hb` (retained, every 30 s), the dashboard's AS3935 panel header was permanently stuck at the static placeholder "Waiting…" because nothing in Node-RED forwarded those topics into Master Dashboard, and the panel only updated on event-driven `as3935_status` messages (which only fire when the chip detects something — never indoors with a 500 kHz selective antenna).

Fix — surface daemon liveness in the panel header

1. **Two new `mqtt in` nodes** subscribe to `lightning/as3935/status` and `lightning/as3935/hb` , datatype JSON, feeding a single `Format AS3935 State` function node.
2. **`Format AS3935 State`** caches each payload in flow context (`as3935_status` , `as3935_hb`) for refresh-replay and forwards typed payloads `{type: 'as3935_ready', …}` and `{type: 'as3935_hb', …}` to Master Dashboard.
3. **`Replay AS3935 State`** (new, fed by the existing 30 s `Refresh Stats` ticker) re-emits cached `flow.as3935_status` and `flow.as3935_hb` so a refreshed dashboard tab repopulates within 30 s — same pattern as the strike-replay fix from May 1.
4. **Master Dashboard** gained two new message handlers and a guard line on the existing `as3935_status` handler:
   - `as3935_ready` → sets the status LED green/amber/red based on calibration (TRCO + SRCO) and `event` field, writes `✓ READY · NF=4 · TUN=10` (or `⚠ CALIB?` / `OFFLINE`) to the header right-side text, and caches the last ready snapshot in `window._as3935Ready` for the heartbeat handler to render.
   - `as3935_hb` → updates the header text every 30 s with current uptime + IRQ count: `✓ READY · NF=4 · up 14m · irq=0` .
   - The existing `as3935_status` handler (disturber/noise) now sets `window._as3935EvtActive = true` for 60 s so a fresh event display isn't immediately overwritten by the next heartbeat; after the timeout it auto-reverts to ready/hb display.

Resulting panel states

Daemon state	Header text	LED

Up, calib OK, idle	`✓ READY · NF=4 · up 14m · irq=0`	🟢 green
Up, calib failed	`⚠ CALIB? · NF=4 · TUN=10`	🟡 amber
Offline (LWT)	`OFFLINE`	🔴 red
Disturber/noise event (60 s display)	`⚠ Disturber 23:18:01`	🟡 amber
Lightning strike	`⚡ N km (energy=…)`	red/amber/green by km
Page refresh while idle	re-populates within 30 s	🟢 green

Layout cleanup

While editing the dashboard template, the `<!-- Event Log -->` block was moved to sit immediately above `<!-- Map -->`. The log is more useful adjacent to the operator's eye-line for the lightning panel than buried at the bottom.

2026-05-08

Lightning Detect — Leaflet map removed

Tab: Lightning Antenna Protector (`75e2cac8ab96f556`) **Node:** Master Dashboard (`557083037f168b22`)

The map was dead weight. Both active strike sources only know *distance*, not direction:

- **Open-Meteo** queries CAPE/ `weather_code` at the home grid point, so `Parse Open-Meteo → Strike` always emits with `lat=home_lat` , `lon=home_lon` .
- **AS3935** chip provides a distance estimate; no azimuth.

Every strike circle therefore stacked on top of the green home marker and was invisible. The Leaflet map was originally designed for a Blitzortung TCP feed (real lat/lon for every strike worldwide), but that integration was never wired up.

What was deleted from the ui_template

- `<div id="leafletMap">` block + the legend bar / `lzCnt` counter
- Leaflet 1.9.4 CSS + JS CDN imports
- CSS rules: `#mapBox` , `#lzWrap` , `.lz-bar` , `.lz-d` , `.lz-home-tip`
- JS state: `lmap` , `homeMarker` , `thrCircle` , `strikeLayer` , the `strikes` array + `MAX=800` cap
- `initMap()` function (Leaflet init, tile layer, threshold ring, home marker, strike layer group)
- `setTimeout(initMap, 400)` boot call + the `resize` listener
- `strikeLayer` / `lzCnt` manipulation in 4 handlers: `strike` , AS3935 `as3935` , `clear` , `strikes_replay`
- The `thrCircle.setRadius()` follow-up in the threshold-update path

Template trimmed from 1,047 lines → 943 lines.

The payload `lat` and `lon` fields in `Strike → Dashboard` are intentionally retained — harmless dead data today, but they let Blitzortung be wired in without re-touching that function (HANDOVER follow-up #7).

Lightning Detect — Nearest Strike gauge persists across page refresh

Nodes: `Strike → Dashboard` (`51367d456e71fb3e`), `Replay on lightning tab` (`f1c57672ba7c0ec1`), Master Dashboard (`557083037f168b22`), `clear all` (`61516f75d2343aae`)

Symptom

The half-circle "Nearest Strike" gauge updated correctly when a strike arrived live, but on every dashboard reload it reverted to the boot value (200 km) and stayed there until the next live strike. The existing 30 s replay tick (which had restored the strike map and event log since 05-01) didn't touch the gauge.

Why

`Strike → Dashboard` was pushing only `{lat, lon, color, ts}` into `flow.strikes` — no `km` field — so even if the replay path had been wired to the gauge, there was nothing to feed it. And the replay emitter (`Replay on lightning tab`) was producing `{type:'strikes_replay', list}` for the map only; no `lastKm` . The boot script also painted `drawGauge(200)` immediately, overwriting the `—` placeholder before any data arrived.

Fix

With the map gone (above), the cache no longer needs a list at all. Simplified to a single value:

- `Strike → Dashboard` : replaced the array push with `flow.set('last_strike_km', km)` .
- `Replay on lightning tab` : now reads `flow.get('last_strike_km')` and emits `{type:'strikes_replay', lastKm:N}` (or returns `null` when there is nothing to replay).
- Master Dashboard: `strikes_replay` handler reduced to `if(typeof d.lastKm === 'number') drawGauge(Math.min(d.lastKm,200));` . Boot-time `drawGauge(200)` removed — gauge now starts at "—" until first strike or replay.
- `clear all` : clears `last_strike_km` (and the legacy `strikes` key, in case anything still reads it).

Verified: trigger synthetic strike → gauge updates → page refresh → gauge redraws to that km within the next 30 s tick. With no cached strike, gauge stays at "—".

Lightning Detect — moved to Shack tab + bypass switch + last-activity recap

Tab moved: Lightning Detect dashboard tab (`dd11372f9c492be8`) — deleted. **New group:** `vu2cpl_grp_lightning` "Lightning Protection" on the Shack tab (`vu2cpl_ui_tab_shack`), width 12, order 9.

The lightning UI is no longer a separate dashboard tab — it lives at the bottom of the main Shack tab. The Master Dashboard ui_template (`557083037f168b22`) was relocated to the new group and trimmed:

- Internal header card (`#hdr` with callsign + clock) removed — Shack tab's existing Header (`eee1a8b8552aa21f`) provides clocks + weather.
- Weather card removed — same reason.
- The wire from `Parse Weather → Header` output 2 → Master Dashboard is no longer needed; weather flows only to the existing Header template now.

Bypass switch

A new vertical `BYPASS` button lives in the switchBox between the ANTENNA and RADIO controls. Off = grey. On = amber with a live `MM:SS` countdown starting at `120:00` . Auto-expires after 2 hours; can be deactivated manually at any time.

While bypass is active:

- A yellow banner above the alert reads `🔕 BYPASS ACTIVE — strikes will alert & log only · no auto-disconnect · expires in MM:SS`.
- Strikes still fire the alert banner (in amber, with text `STRIKE N km — BYPASS · alert only`) but **Trigger Disconnect** (`d62fb0c3c40f03b7`) early-outs without publishing MQTT off, without flipping `flow.antenna_off`, and without starting the reconnect timer.
- The dashboard ANTENNA + RADIO switches stay green ON.
- Activating bypass also force-reconnects (cancels any pending Reconnect Timer + sends an ON command to ant + radio) — designed for "I'm on the air, the storm is far enough away, don't drop me".

State variables: `flow.bypass_active` (bool), `flow.bypass_expires_at` (ms timestamp). Init Defaults (`ec1fd4dece8c4dc0`) clears both on every deploy/launch — bypass never survives a Node-RED restart, by design.

Server side, three new nodes (Lightning Antenna Protector tab):

- `POST /lightning/bypass` http-in — receives `{action:'on'|'off'}`
- `Bypass Handler` function (3 outputs):
  1. → Master Dashboard — emits `{type:'bypass_state', active, expiresAt}` plus a log line
  2. → Reconnect Timer — `{payload:'cancel'}` to kill any pending timer
  3. → Execute Reconnect — force ant + radio ON immediately on activation
- `200 OK` http response (parallel wire from http-in for immediate ack)

The `Replay on lightning tab` function (`f1c57672ba7c0ec1`) now emits `bypass_state` on every Refresh Stats tick (30 s), so the banner + countdown survive page refresh. Includes a server-side expiry safety net in case the in-process `setTimeout` is somehow lost.

Reconnect buttons relabelled

The two ↺ icons next to ANTENNA and RADIO were tiny 36×36 squares with no text. Now they stretch to match the switch row height with `↺ RECONNECT` text and a 110 px minimum width — much easier to spot.

Last-activity recap (replaces auto-clear)

Old behaviour: 30 s after every alert, the banner displayed `✔ No recent activity` — misleading, since plenty of activity had just happened. New behaviour:

- On boot, before any alert: `⏱ Awaiting first event` (muted).
- During an alert: full colour (red / amber / green / yellow-bypass).
- 30 s after the alert: banner transitions to muted recap form `⏱ Last: STRIKE 12 km — ANTENNA + FlexRadio OFF · 4m ago`.
- The `Nm ago` text auto-refreshes every 30 s. Hours roll into `2h 17m ago`, days into `1d ago`.

Two `window.lastAlert` tracking lines + a `setInterval` driver in the ui_template; no flow node changes required.

Strike → Dashboard simplification

Removed the `flow.set('last_strike_km', km)` line — the gauge that consumed it was already gone in `bf1b941`, so the write was dead.

Verified end-to-end

- Bypass-OFF + TEST inject (6 km strike) → red alert + ant/radio → OFF + reconnect timer starts. ✓

- Bypass-OFF + Open-Meteo poll (CAPE → synthetic 0–20 km strike) → same as TEST. ✓
- Bypass-ON + TEST inject → amber alert "BYPASS · alert only", switches stay green, no MQTT publish. ✓
- Bypass-ON + Open-Meteo poll → same as bypass-ON TEST (verified during a moderate-CAPE day where polls fired every 5 min and TD's status badge stayed `BYPASS · Open-Meteo Nkm — disconnect skipped` for the full 2 h). ✓
- Page refresh during bypass → banner + button + countdown all replay within 30 s via Refresh Stats. ✓

2026-05-09

LP-700 — switched from direct USB-HID to lp700-server WebSocket gateway

Tab: LP-700-HID ws (`18fb42443172f33c`) — renamed from `LP-700-HID` , trimmed from 25 nodes to 18.

VU3ESV's LP-700-Server is now running as `lp700-server.service` on the Pi at `ws://192.168.1.169:8089/ws` . Pure-Go single binary; owns `/dev/hidraw*` ; multi-client WebSocket fan-out.

Why migrate

`@gdziuba/node-red-usbhid` only allows one process to hold the LP-700's HID handle at a time, which would block any other consumer (browser, phone, the planned Mac SwiftUI app). The Go gateway moves ownership to a dedicated service; Node-RED becomes one of N clients on the bus, same as everything else.

Deployment was already done — the user installed `lp700-server` on the Pi and confirmed `/healthz` before the Node-RED migration. This entry covers only the Node-RED side.

What was deleted from the LP-700 tab

11 nodes, all direct-HID plumbing:

- `getHIDdevices` , `HIDdevice (HID-LP)` — HID node config + worker
- `Poll Meter Values` , `LP Dice and Slice` — poll trigger + binary parser
- `inject Poll Devices` , `inject Kickstart` , `trigger` — startup machinery
- `Raw Buffer` , `Errors` — debug nodes
- `CH cmd` , `RNG cmd` — HID-buffer button payloads (b[1]=56 / b[1]=57)

What was added (imported from `examples/node-red/lp700-websocket-flow.json`)

13 nodes from VU3ESV's example flow, all namespaced `lp7…` to avoid ID collisions:

- `ws://lp700-server/ws` — websocket-in (URL `ws://192.168.1.169:8089/ws` , `wholemsg=false` so payload arrives as message). One websocket-client config (`lp7wsclient00001`).
- `ws://lp700-server/ws` — websocket-out (same client config, send side)
- `Parse frame` — JSON decoder; routes telemetry to output 1, heartbeat to output 2, ack to output 3
- `Reshape (KD4Z-compatible)` — maps the gateway's verbose snapshot to the flat `power_avg / power_peak / swr / scale / mode / channel / range` keys that the existing `LP State Aggregator` (and KD4Z's upstream nodes) expect. Zero changes downstream.
- `Connection state` — exposes link status as a status badge
- `ack` debug — shows server replies to control verbs (`{ok:true}` or `control disabled` etc.)

- 7 `inject` nodes labelled with each verb (peak/channel/range/alarm/ mode/setup/freeze) — kept for canvas-side testing; deletable later
- 2 comment nodes labelling the inbound/outbound halves of the gateway

What was kept and rewired

- `LP State Aggregator` — function body unchanged; now fed by `Reshape` output (was `LP Dice and Slice` output)
- `LP-700 Panel` — ui_template unchanged
- `LP Button Router` — function body rewritten. Old version produced HID buffers (`Buffer.alloc(64)` with `b[1]=56` for channel, `b[1]=57` for range). New version emits JSON command frames:

  ```
  if (msg.topic === 'lp700_ch')
    return { payload: JSON.stringify({type:'command', id:'…',
  action:'channel_step'}) };
  if (msg.topic === 'lp700_rng')
    return { payload: JSON.stringify({type:'command', id:'…',
  action:'range_step'}) };
  ```

  Wired directly into the websocket-out node (no link-in/out indirection since the gateway and dashboard live on the same tab).

Configuration gotcha

First import attempt produced a fake-green connection — the imported `websocket-client` config had a stale client ID that didn't match the URL after editing. Resolved by deleting the imported config and creating a fresh one pointed at `ws://192.168.1.169:8089/ws` . Telemetry flowed within 2 s of redeploy.

Deps that became optional

- npm: `@gdziuba/node-red-usbhid 1.0.3` — still installed but no longer used by any flow. Keep for now; uninstall after a week of stable WS operation.
- apt: `libudev-dev` , `librtlsdr-dev` , `libusb-1.0-0-dev` — only needed to *build* the HID node. Same retention guidance.

Verified

- Telemetry frames arriving at ~25 Hz (matches LP-700 LCD update cadence)
- Power/SWR/channel/range labels all live on the dashboard
- CH and RNG buttons step the meter; `ack` debug shows `{ok:true}`
- Service status: `systemctl status lp700-server` clean; no Node-RED HID errors after restart

Post-deploy fix (08c907f)

After the migration deployed, the dashboard panel was stuck displaying stale `flow.lpState` values (AVG/PEAK 14 W, SWR 1.25 — the last numbers the old `LP Dice and Slice` had cached before the migration). Telemetry was flowing fine through the gateway → Reshape pipeline; the bug was that **`LP State Aggregator` couldn't see the new shape**.

KD4Z's `LP Dice and Slice` returned `{power_avg, power_peak, swr, …}` at the **top level of msg**. The aggregator was hand-written to read `msg.power_avg` directly. VU3ESV's `Reshape (KD4Z-compatible)` puts those same keys on **`msg.payload`** (per his README, matching how KD4Z's

downstream nodes consume them). So `msg.power_avg` was always `undefined`, every conditional in the aggregator was false, and `flow.lpState` never updated — the panel kept emitting whatever values were in flow context at deploy time.

Fix: aggregator function body now reads from `msg.payload` first, falling back to the top of `msg` if payload is absent. Backward- compatible with the legacy KD4Z shape.

```
const d = (msg.payload && typeof msg.payload === 'object') ? msg.payload : msg;
// …then read d.power_avg, d.power_peak, d.swr, etc.
```

Lesson for future migrations on this flow: when swapping a parser, audit every consumer that reads from `msg` directly (not `msg.payload`) — the ui_template and the aggregator can both quietly drift out of sync with the new payload shape.

Pi-side scripts — check in `rpi_agent.py` + `monitor.sh` + systemd unit

Discovered during a CLAUDE.md audit that the "RPi Fleet Monitor" docs described `rpi_agent.py` as both the HTTP reboot/shutdown handler AND the MQTT telemetry publisher. In reality the script is HTTP-only — telemetry comes from a separate shell script (`monitor.sh`) running once a minute via the `vu2cpl` user crontab. Both scripts existed only on the Pis themselves, not in this repo. If a Pi ever needed re-imaging the operational glue would have been lost.

Files added (root level)

- **`rpi_agent.py`** — 21-line stdlib-only `BaseHTTPRequestHandler` bound to `:7799`. Two routes: `POST /reboot` and `POST /shutdown`, each shells out to `sudo reboot` / `sudo shutdown -h now`. Returns `{"status": "rebooting"}` JSON 200 then forks the syscall.
- **`monitor.sh`** — bash, executable. Reads CPU (`top -bn1`), mem (`free`), temp (`/sys/class/thermal/thermal_zone0`), disk (`df /`), uptime (`uptime -p` with `/proc/uptime` fallback), IP (`hostname -I` first non-blank). Publishes each as a separate MQTT topic `rpi/<hostname>/{cpu,mem,temp,disk,uptime,ip,status}` to broker `192.168.1.169` via `mosquitto_pub`.
- **`rpi-agent.service`** — systemd unit. Runs as user `vu2cpl`, `Restart=always`, `After=network.target`. Standard install path `/etc/systemd/system/rpi-agent.service`.

CLAUDE.md changes

- Replaced the 7-line "RPi Fleet Monitor" subsection with a split "HTTP control agent" + "Telemetry publisher" + "Home Assistant Pi (special case)" structure. Each agent gets explicit per-Pi install commands (cp, daemon-reload, enable --now, sudoers, crontab line).
- Backup section: extended `tar -czf` glob to include the not-yet- checked-in scripts (`fetch_clublog.sh`, `enable_file_context.sh`, `power_spe_on.py`) flagged with HANDOVER follow-up reference.
- Added a "Pi-side scripts already in this repo" table mapping each repo file to its canonical deployment path under `/home/vu2cpl/`.

Sudoers entry confirmed

```
vu2cpl ALL=(ALL) NOPASSWD: /sbin/reboot, /sbin/shutdown
```

documented at `/etc/sudoers.d/rpi-agent` in the install commands.

Pi-side scripts — `power_spe_on.py` checked in, `fetch_clublog.sh` retired

Follow-up to the earlier "Pi-side scripts" entry above.

- **`power_spe_on.py`** added to the repo. Tiny FTDI DTR/RTS toggling helper used to soft-power the SPE Expert 1.5 KFA. Already referenced in CLAUDE.md ("Power-on requires external Python: `python3 ~/power_spe_on.py`") but never checked in. Now in repo root with the rest.
- **`enable_file_context.sh`** was discovered to already be tracked in the repo — the earlier "not yet checked into repo" annotation in CLAUDE.md was wrong. Annotation removed.
- **`fetch_clublog.sh`** has been deleted from the Pi entirely. The output file `nr_dxcc_live.json` it produced is referenced by zero nodes in `flows.json` (verified by full-flow grep). Live DXCC fetch happens in-flow via `Build Club Log API Request` → `Fetch All Modes + Parse` → writes to `nr_dxcc_seed.json`. The shell script and its output were a vestige of an older workflow. Removed: `~/fetch_clublog.sh`, `~/bin/`, the orphaned `nr_dxcc_live.json`. Cron entry checked and absent. Club Log password + API key rotated since they had been pasted in plaintext during the audit.

CLAUDE.md updates:

- "Pi-side scripts already in this repo" table extended to include `power_spe_on.py` and `enable_file_context.sh`.
- BACKUP section's tar glob trimmed: `fetch_clublog.sh` and `enable_file_context.sh` no longer listed (one's deleted, one's now in repo). The `power_spe_on.py` line drops the "not yet checked into repo" annotation.

HANDOVER follow-up #9 closed (the deferred Pi-side script migration).

`DEPLOY_PI.md` — full Pi onboarding runbook

Drafted while preparing to deploy the agent + telemetry to the two remaining Pis (and eventually the HA Pi). The CLAUDE.md "RPi Fleet Monitor" subsection has the right copy-paste blocks but lacks verification, troubleshooting, and the special-cases / removal flows.

`DEPLOY_PI.md` covers the full per-Pi lifecycle:

1. Prerequisites (SSH, hostname uniqueness, broker reachability)
2. `scp` the three files from the Mac repo
3. Install at canonical paths with the `sudo cp + chown` ownership gotcha (the 2026-05-07 quirk) called out
4. mosquitto-clients install + smoke-test telemetry with `mosquitto_sub`
5. Idempotent crontab append (`grep -v 'monitor.sh'` first)
6. `/etc/sudoers.d/rpi-agent` with `visudo -c` syntax check
7. `systemctl enable --now rpi-agent` + `curl /probe` smoke-test
8. Add hostname to `httpDevices` in Node-RED → deploy → `nrsave`
9. Dashboard verification

Plus: HA Pi special case (HA-side automation, not script-based), troubleshooting table (8 common failure modes), and a clean decommission flow.

CLAUDE.md gets a one-line cross-reference at the top of the RPi Fleet Monitor section pointing to the new doc.

Pi GPS NTP Server — new standalone repo, stratum 1 in service

Standalone repo: vu2cpl/pi-gps-ntp-server

The shack now has a dedicated GPS-disciplined NTP server. A previously unused Raspberry Pi 3B was repurposed as `gpsntp.local` (192.168.1.158) running stratum-1 NTP for the LAN.

Pivot from the original ESP32 plan

The project directory was originally `~/projects/ESP32 GPS NTP/` and scoped as an embedded firmware build (ESP32 + NEO-M8N + custom NTP code). After re-evaluation the path was changed to a Raspberry Pi running chrony + gpsd + kernel PPS. Reasoning recorded in detail in the new repo's `HANDOVER.md`; short version:

- chrony is what most public stratum-1 servers actually run; ESP32 hobby code has to re-implement leap seconds, holdover, multi-source comparison, and stratum honesty — chrony already does this correctly.
- Linux kernel timestamps NTP packets at NIC level; ESP32 lwIP does this in userspace. Client-visible accuracy ends up better on the Pi.
- Wired Ethernet by default vs Wi-Fi NTP jitter swamping ESP32's local PPS lock.
- Debug with SSH + `chronyc` + `gpsmon` rather than serial console + logic analyzer.

Local directory renamed `~/projects/ESP32 GPS NTP/` → `~/projects/Pi GPS NTP Server/`.

How the GPS is sourced

No new GPS hardware was purchased. The existing **QRP Labs QLG1** (MediaTek chipset, 10 ns RMS PPS) that already feeds the U3S beacon was tapped via its unused **6-way connector**. The U3S retains its own connection on the 4-way header and is electrically undisturbed.

QLG1 outputs are 5 V (74ACT08 buffers from +5 V rail). Pi GPIO is not 5 V tolerant. Each tapped signal (TXD, PPS) goes through a 2.2 kΩ + 3.3 kΩ voltage divider that drops 5 V → 3.0 V — comfortably above the Pi's input-high threshold (~2.3 V) and well below the 3.6 V absolute max. Three wires from QLG1 6-way pins 3 (TXD), 5 (1PPS), 6 (GND) to Pi physical pins 10 (GPIO15 / UART RX), 12 (GPIO18), 6 (GND).

Software stack

- **Pi OS Lite, 64-bit, Trixie** (Bookworm successor — current Imager default). Headless install via Imager OS Customisation with hostname `gpsntp` and SSH public-key auth.
- `/boot/firmware/config.txt` adds: `enable_uart=1`, `dtoverlay=disable-bt` (frees PL011 from BT for GPIO14/15), and `dtoverlay=pps-gpio,gpiopin=18`.
- Serial login console disabled via `raspi-config`.
- **`gpsd 3.25`** parsing NMEA from `/dev/serial0`, exporting PPS via SHM-2. `GPSD_OPTIONS="-n"` so it polls without waiting for clients (critical — without this chrony never gets data).
- **Kernel PPS** via `pps-gpio` overlay on GPIO18 → `/dev/pps0`.
- **chrony** with:
  - `refclock SHM 0 refid NMEA offset 0.0 delay 0.2 noselect` — gpsd's coarse second-of-time, used only to label which integer second the PPS edge belongs to.
  - `refclock PPS /dev/pps0 lock NMEA refid PPS` — kernel PPS device, locked to NMEA's second-numbering. The actual time source.
  - `allow 192.168.1.0/24` and `allow fd00::/8` (IPv6 ULA, so `sntp gpsntp.local` from the Mac doesn't get a "recv failure" on its first IPv6 attempt before falling back to IPv4).

Achieved performance (first lock)

Metric	Value
Reference ID	PPS
Stratum	1
PPS error	±152 ns
System clock vs GPS truth	35 ns slow
Skew	0.009 ppm
Root dispersion	18 µs

Mac (`MiniM4-Pro`) configured as client via `sudo systemsetup -setnetworktimeserver gpsntp.local` . `timed` disciplines the Mac continuously; observed offset 1–4 ms on the LAN (limited by macOS scheduling jitter, not the Pi or the network).

SkimServer Mac, the U3S, and any other shack PCs all gain a precise, LAN-local, internet-independent time source by way of the Mac (or by pointing them at `gpsntp.local` directly).

Repo contents

- `BUILD.md` — full step-by-step procedure with per-stage verification and a detailed troubleshooting section.
- `HANDOVER.md` — context, decisions made, ops-checks block for future diagnosis.
- `README.md` — landing page with perf headlines and the why-Pi-not-ESP32 case.
- `LICENSE` — MIT, © 2026 Manoj Kumar R (VU2CPL).

Known risk parked

When the U3S transmits, the QLG1 sits next to the transmitter and may RFI-desensitize. This is a pre-existing condition of the U3S+QLG1 combination; the new NTP server is downstream of it. If `chronyc tracking` shows fix-loss during transmissions over the next week or two, fall back to a dedicated NEO-M8N + antenna for the Pi (path documented in `pi-gps-ntp-server/HANDOVER.md`). HANDOVER follow-up #10.

---

gpsntp — first Pi onboarded via `DEPLOY_PI.md`

Companion to the GPS NTP server above, and the first real-world exercise of the new `DEPLOY_PI.md` runbook. The `gpsntp` Pi was added to the RPi Fleet Monitor agent set:

- `rpi_agent.py` deployed to `/home/vu2cpl/` . Active on :7799, HTTP 404 probe verified.
- `monitor.sh` deployed to `/home/vu2cpl/` . Per-minute crontab line `* * * * * /home/vu2cpl/monitor.sh` publishes 7 MQTT topics (`cpu/mem/temp/disk/uptime/ip/status`) under `rpi/gpsntp/` to broker @ 192.168.1.169.
- `rpi-agent.service` deployed to `/etc/systemd/system/` . Enabled + active.
- Sudoers entry `/etc/sudoers.d/rpi-agent` : `vu2cpl ALL=(ALL) NOPASSWD: /sbin/reboot, /sbin/shutdown` .

Smoke test (`mosquitto_sub -h 192.168.1.169 -t "rpi/gpsntp/#" -v -C 7`) caught all 7 messages cleanly. Important note for `DEPLOY_PI.md` follow-up: `monitor.sh` publishes without `-r` (retain), so the smoke-test subscriber must be subscribed *before* `monitor.sh` runs — otherwise the messages fly past the broker and the sub hangs waiting for retained data that isn't there. Worth folding into the runbook so the next Pi onboarding doesn't get tripped by it.

The Node-RED `httpDevices` map (function `a0695975fec84e2c`) still needs `'gpsntp':'http://gpsntp.local:7799'` added so the Reboot/Shutdown buttons surface for the new host. Tracked as HANDOVER follow-up #9.

Standard Commit Sequence (reminder)

Per CLAUDE.md rule #4, extract the DXCC Tracker tab alongside flows.json:

```
cd ~/.node-red/projects/vu2cpl-shack
python3 -c 'import json; d=json.load(open("flows.json")); v=[n for n in d if
n.get("z")=="d110d176c0aad308" or n.get("id")=="d110d176c0aad308"];
json.dump(v,open("clublog_dxcc_tracker_v7.json","w"),indent=2)'
git add flows.json clublog_dxcc_tracker_v7.json
git commit -m "<description>"
git push
```

After push, on the Pi:

```
cd ~/.node-red/projects/vu2cpl-shack
git pull
sudo systemctl restart nodered
```